Forex Exchange Data in C

This presentation outlines a C program designed to handle Forex exchange data. It includes defining a structure for holding currency pair information, bid and ask prices, last price, and timestamp. The program also provides functions to create, print, and free ForexData instances, ensuring efficient memory management and data handling.

by Tsepo Maleke

Defining the ForexData Structure

Currency Pair

A character array to store the currency pair (e.g., "EURUSD").

Bid Price

A double to store the current bid price.

Ask Price

A double to store the current ask price.

Last Price

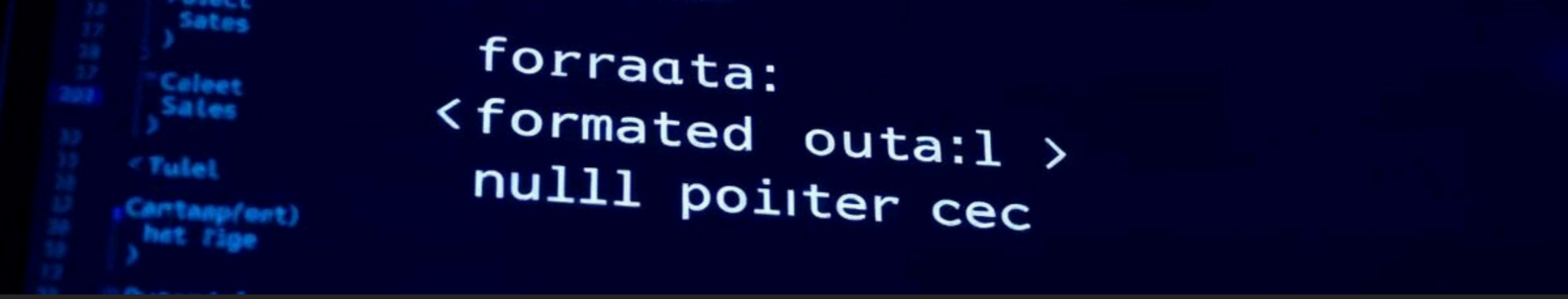
A double to store the last traded price.

The **ForexData** structure is defined to hold key information about Forex exchange rates. It includes fields for the currency pair, bid price, ask price, last price, and timestamp. This structure is crucial for organizing and managing Forex data efficiently.

Creating a New ForexData Instance

```
ForexData* createForexData(const char* pair, double bid, double ask, double last, const char* time) {
    ForexData* newData = (ForexData*)malloc(sizeof(ForexData));
    if (newData == NULL) {
        fprintf(stderr, "Memory allocation failed\n");
        exit(1);
    }
    strncpy(newData->currencyPair, pair, sizeof(newData->currencyPair) - 1);
    newData->currencyPair[sizeof(newData->currencyPair) - 1] = '\0';
    newData->bidPrice = bid;
    newData->askPrice = ask;
    newData->lastPrice = last;
    strncpy(newData->timestamp, time, sizeof(newData->timestamp) - 1);
    newData->timestamp[sizeof(newData->timestamp) - 1] = '\0';
    return newData;
}
```

The **createForexData** function allocates memory for a new **ForexData** instance and populates it with the provided data. It includes error handling to ensure memory allocation is successful and uses **strncpy** to prevent buffer overflows when copying strings.



Printing ForexData

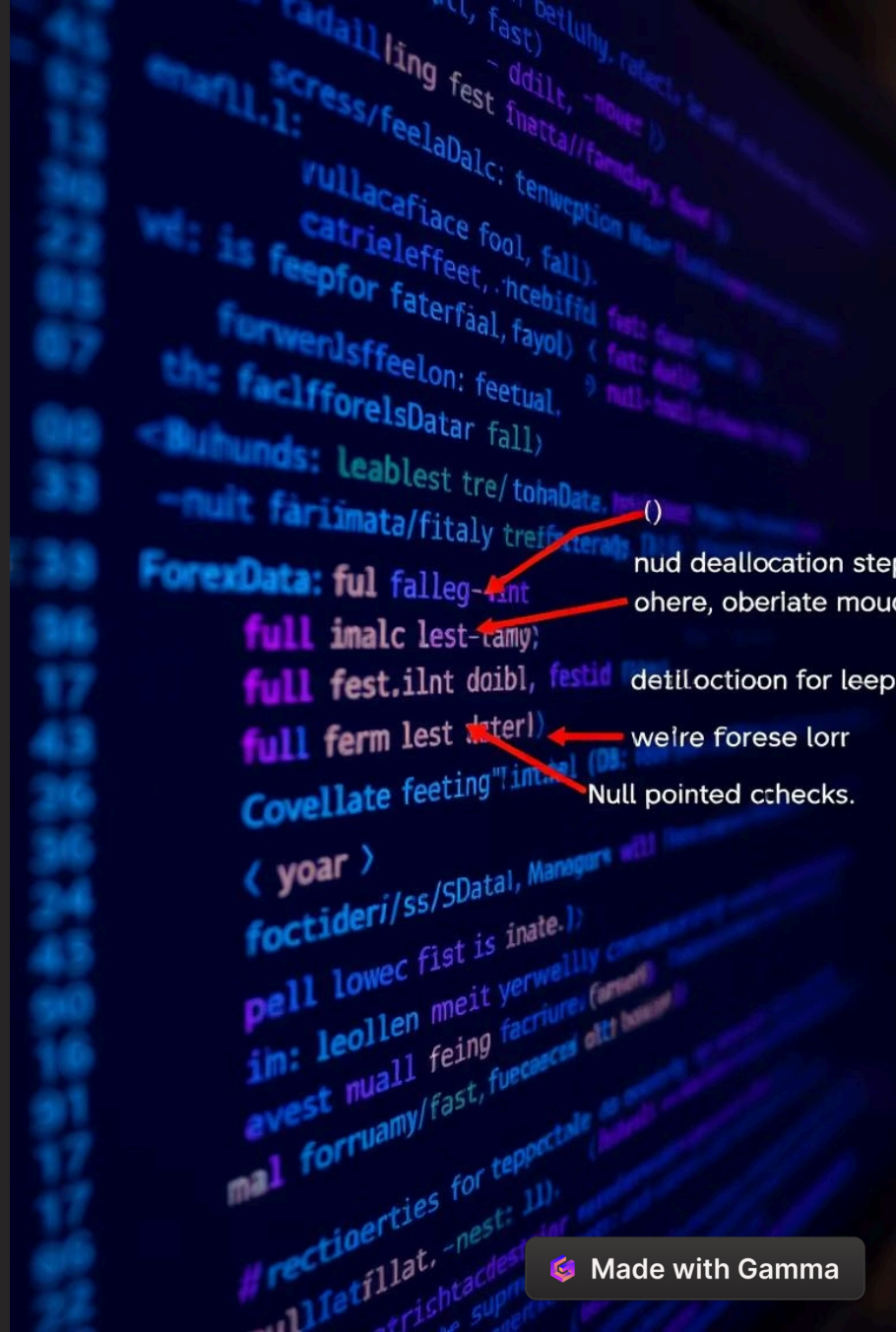
```
void printForexData(const ForexData* data) {
    if (data != NULL) {
        printf("Currency Pair: %s\n", data->currencyPair);
        printf("Bid Price: %.5f\n", data->bidPrice);
        printf("Ask Price: %.5f\n", data->askPrice);
        printf("Last Price: %.5f\n", data->lastPrice);
        printf("Timestamp: %s\n", data->timestamp);
    }
}
```

The **printForexData** function prints the contents of a **ForexData** instance to the console. It includes a null pointer check to ensure that the function does not attempt to access invalid memory. The output is formatted to display the currency pair, bid price, ask price, last price, and timestamp.

Freeing ForexData

```
void freeForexData(ForexData* data) {
    if (data != NULL) {
        free(data);
    }
}
```

The **freeForexData** function deallocates the memory occupied by a **ForexData** instance. It includes a null pointer check to ensure that the function does not attempt to free invalid memory. This function is crucial for preventing memory leaks.



Main Function: Example Usage

```
int main() {
    // Example usage
    ForexData* data = createForexData("EURUSD", 1.12345, 1.12355,
1.12350, "2025-04-11 10:00:00");
    printForexData(data);
    freeForexData(data);
    return 0;
}
```

The **main** function demonstrates how to use the **createForexData**, **printForexData**, and **freeForexData** functions. It creates a **ForexData** instance, prints its contents, and then frees the allocated memory. This example showcases the complete lifecycle of a **ForexData** instance.

Memory Management

1 Allocation

Use **malloc** to allocate memory for **ForexData** instances.

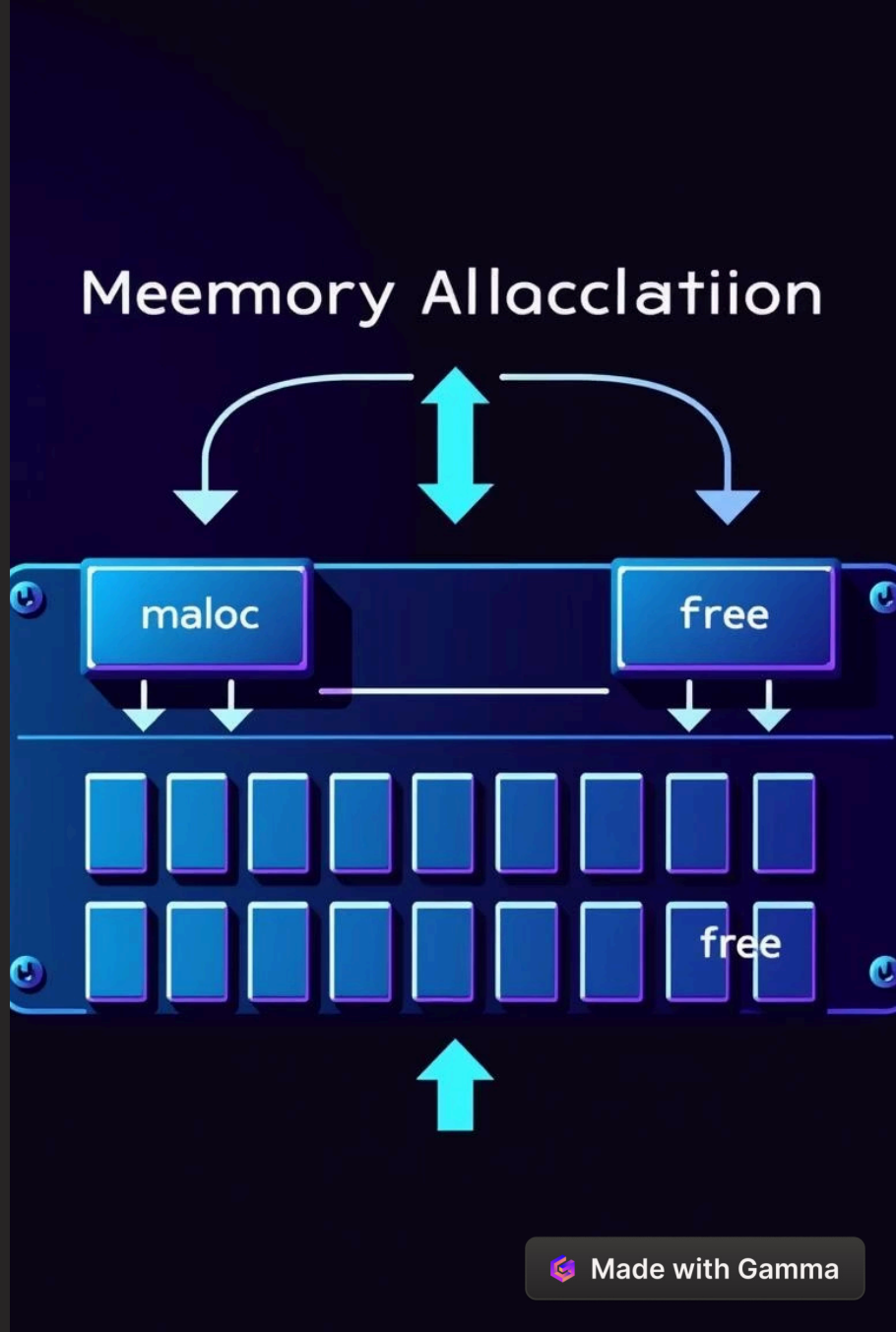
2 Deallocation

Use **free** to deallocate memory when **ForexData** instances are no longer needed.

3 Error Handling

Check for null pointers to prevent memory access errors.

Proper memory management is crucial in C to prevent memory leaks and ensure program stability. The program uses **malloc** to allocate memory for **ForexData** instances and **free** to deallocate memory when it is no longer needed. Error handling is implemented to check for null pointers and prevent memory access errors.



String Handling

strncpy

Use **strncpy** to prevent buffer overflows when copying strings.

Null Termination

Ensure strings are null-terminated to prevent reading beyond allocated memory.

String handling in C requires careful attention to prevent buffer overflows and memory access errors. The program uses **strncpy** to copy strings and ensures that all strings are null-terminated. This helps to prevent reading beyond the allocated memory and ensures program stability.

Error Handling

1

Memory Allocation

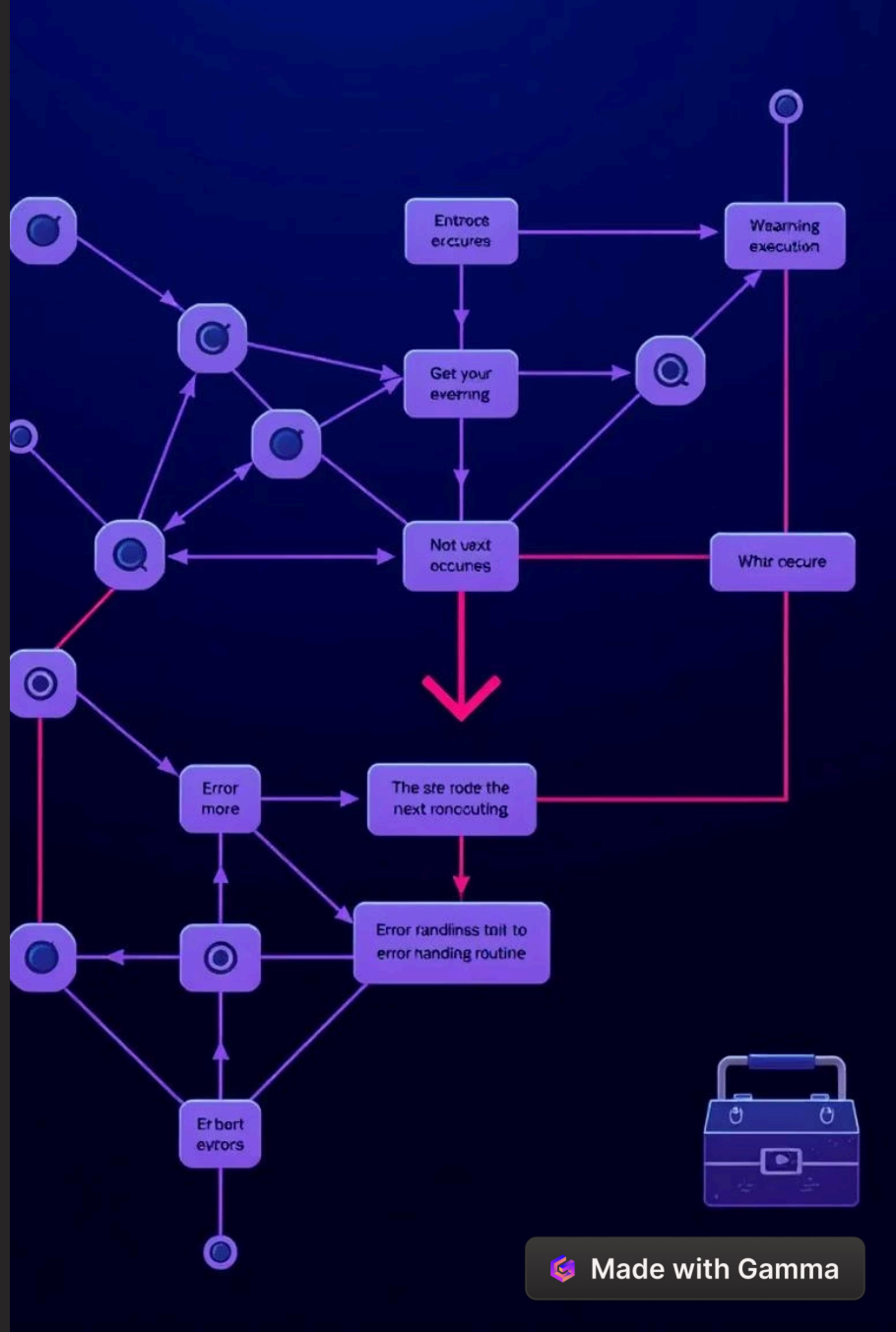
Check if **malloc** returns **NULL**.

2

Null Pointers

Check if pointers are **NULL** before dereferencing.

Robust error handling is essential for creating reliable C programs. The program checks if **malloc** returns **NULL** to handle memory allocation failures. It also checks if pointers are **NULL** before dereferencing them to prevent segmentation faults.



Key Takeaways

ForexData Structure

Defined to hold Forex exchange data.

Memory Management

Use **malloc** and **free** for memory allocation and deallocation.

String Handling

Use **strncpy** and null termination to prevent buffer overflows.

This presentation covered the key aspects of handling Forex exchange data in C. It included defining the **ForexData** structure, creating functions for memory management, and implementing robust error handling. These techniques are crucial for developing reliable and efficient C programs.